



UT1 — Introducció a Spring

CIFP Pau Casesnoves · IFC33C · Curs 2026–2027

Mòdul: 0613. Desenvolupament web en entorn servidor

Professor: David Pons

Presentació

Una aplicació web de servidor està formada per objectes que col·laboren: un controlador rep peticions, un servei executa les operacions i altres components aporten les dades o les eines necessàries. Per entendre com funciona l'aplicació, hem de saber qui crea aquests objectes, com es connecten i durant quant de temps es reutilitzen.

En aquesta unitat estudiem com Spring gestiona els objectes i les seves dependències. Els exemples acompanyen el code along: construïm les classes, explicam les decisions i observam el resultat mitjançant peticions HTTP i missatges de diagnòstic.

Els exemples segueixen el fil del code along: motors i dependències, comptadors i tickets, carrets amb una factory, peticions i sessions, inicialització diferida, cicle de vida i consulta de l'hora del servidor.

En acabar, has de poder seguir una petició des del controlador fins als objectes que utilitza, justificar quina dependència s'injecta i predir quan es crea, es reutilitza i es finalitza cada bean. Comprovar una resposta és el punt de partida; explicar-la a partir del codi és l'objectiu.

Els fragments mostren les classes o les parts que expliquen cada concepte. Quan s'ometen el paquet, els imports o altres parts del controlador, s'han d'afegir segons l'organització del projecte. Cada classe pública va en un fitxer amb el mateix nom. Les variants introductòries permeten comparar comportaments; el text indica quan tornam a la configuració emprada a classe.

La referència és el projecte `unitat1_2627`. Conservem els noms dels seus fitxers, inclosos `Unitat1Contoller`, `ShoppingCartService` i `ShoppingCartFactory`. Les variants d'una mateixa classe substitueixen temporalment la versió anterior: no s'han d'afegir com a classes duplicades. Els casos d'altres biblioteques i la connexió amb la pràctica d'informes són exemples d'aplicació dels conceptes, no components presents en aquest projecte.

Índex

1. Spring Framework, Spring Core i Spring Boot.
2. Inversió de control: IoC.
3. Injecció de dependències: DI.
4. Beans i contenidor de Spring.
5. Scopes: quantes instàncies hi ha i quan es reutilitzen.
6. Inicialització diferida: @Lazy.
7. Cicle de vida: @PostConstruct i @PreDestroy.
8. Configuració explícita: @Configuration i @Bean.
9. Relació entre els conceptes i els exemples.

1. Spring Framework, Spring Core i Spring Boot

Spring Framework és un conjunt de llibreries Java que facilita la construcció d'aplicacions. Ofereix suport per a diferents necessitats: gestionar objectes, atendre peticions web, treballar amb dades o coordinar transaccions, entre d'altres.

La seva base és el contenidor IoC, que estudiem dins **Spring Core**. Aquest contenidor crea i configura els objectes de l'aplicació que li hem declarat i hi proporciona les dependències que necessiten. Dos conceptes expliquen aquesta manera de treballar: la **inversió de control**, o IoC, i la **injecció de dependències**, o DI.

Anomenem **bean** un objecte gestionat per aquest contenidor. Per exemple, `@Component` permet que Spring detecti una classe pròpia i en registri una definició de bean. La classe és el tipus; el bean és l'objecte gestionat. Amb aquesta distinció ja podem llegir els primers exemples; a l'apartat 4 veurem les formes de declarar-los.

Spring Boot facilita la preparació i l'arrencada d'aplicacions basades en Spring. Aporta configuració automàtica segons les dependències i la configuració de l'aplicació, i agrupa dependències habituals en starters. A l'aplicació web que desenvolupem també facilita executar un servidor integrat.

Element	Paper en els exemples de la UT1
Spring Framework	Proporciona el framework amb què construïm l'aplicació.
Spring Core i el contenidor IoC	Gestionen beans, dependències, scopes i inicialització.
Spring MVC	Relaciona les peticions HTTP amb els mètodes dels controladors.
Spring Boot	Prepara i arrenca l'aplicació amb una configuració inicial adequada.

La classe d'arrencada de l'aplicació és:

```
@SpringBootApplication
public class Unitat12627Application {
    public static void main(String[] args) {
        SpringApplication.run(Unitat12627Application.class, args);
    }
}
```

`SpringApplication.run(...)` arrenca l'aplicació i crea el context de Spring.

`@SpringBootApplication` activa, entre altres funcions, la configuració automàtica i la cerca de components. Per defecte, aquesta cerca parteix del paquet de la classe d'arrencada i inclou els seus subpaquets. Per això situem `controller`, `service`, `domain` i `config` davall el paquet principal.

Una primera resposta HTTP permet comprovar que l'aplicació atén peticions. Aquest és el fragment inicial d'`Unitat1Controller`, sense les dependències i les altres rutes que hi afegim:

```
@RestController
public class Unitat1Controller {
    // S'ometen les dependències i les altres rutes.
    @GetMapping("/")
    public String holaMon() {
        return "hola mon";
    }
}
```

`@RestController` declara un controlador que escriu el resultat del mètode al cos de la resposta. `@GetMapping("/")` relaciona una petició HTTP GET a / amb `holaMon()`. Si executam el projecte amb el port habitual, podem visitar `http://localhost:8080/`.

En aquests primers exemples retornam text. La lògica que genera la resposta s'executa al servidor; el navegador en rep el resultat.

El projecte i el recorregut d'una petició

El `pom.xml` del projecte declara Java 25, Spring Boot 4.1.1 i el starter de Spring Web MVC. Cal obrir `unitat1_2627` com a projecte Maven, emprar un JDK compatible amb aquesta configuració i executar `Unitat12627Application`. La configuració actual només fixa el nom de l'aplicació; no activa lazy global ni canvia el port per defecte.

```
Navegador -- GET /cotxe --> servidor web integrat
-> Spring MVC localitza el mètode associat a /cotxe
-> Unitat1Controller.conduirCotxe()
-> CotxeService.conduir() -> Cotxe.engega() -> Motor.engega()
-> el text retornat s'envia al navegador com a resposta HTTP
```

Paquet dins <code>cat.paucasesnovescifp.unit at1_2627</code>	Què hi trobam?
controller	Rutes HTTP i delegació als serveis.
service	Operacions que utilitzen els components.
domain	Objectes dels exemples: cotxe, motors, comptadors, tickets i altres components.
config	Configuració explícita del formatador de data i hora.

En aquest projecte introductori no hi ha repositoris, base de dades ni plantilles HTML. El paquet `domain` agrupa també components de demostració: estar dins aquest paquet no converteix una classe en una entitat persistent. `@RestController` retorna aquí text, no una vista ni necessàriament JSON.

Per observar cada exemple, mantenim obertes la consola de l'aplicació i la resposta del navegador. Repetir una petició conserva el context; reiniciar l'aplicació el torna a crear i reinicia l'estat dels exemples. Si modifiquem una anotació, reiniciem abans de comparar els resultats.

2. Inversió de control: IoC

2.1. Crear les dependències dins l'objecte

Una **dependència** és un objecte que una classe necessita per fer la seva feina. Un cotxe, per exemple, necessita un motor.

Sense separar la creació dels objectes, podríem escriure:

```
public class MotorBenzina {
    public String engega() {
        return "Motor benzina engega";
    }
}

public class Cotxe {
    private final MotorBenzina motor;

    public Cotxe() {
        this.motor = new MotorBenzina();
    }

    public String engega() {
        return "Engegant motor ->" + motor.engega();
    }
}
```

Cotxe decideix quina implementació de motor utilitza i també la construeix. Si volem passar a un motor elèctric, hem de modificar la classe. Per provar el cotxe amb un motor de prova, tampoc no tenim una manera directa de proporcionar-li aquest objecte.

El problema és l'**acoblament** entre el comportament del cotxe i la creació d'una dependència concreta.

2.2. Rebre les dependències des de fora

La **inversió de control** és un principi de disseny pel qual delegam en un altre component una responsabilitat que l'objecte controlava directament. En el cas que estudiem, delegam la creació i la connexió de les dependències.

El cotxe passa a rebre un motor des de fora. Amb Spring, aquesta feina la fa el **contenedor IoC**, habitualment a través d'un `ApplicationContext`: coneix les definicions dels beans, crea els objectes quan correspon i resol les seves dependències.

La idea es pot representar així:

```
Creació interna: Cotxe construeix MotorBenzina
Amb IoC i DI:   Spring proporciona un Motor a Cotxe
```

Separar aquestes responsabilitats permet canviar la configuració sense reescriure la lògica del cotxe i facilita provar cada classe. Això no elimina les dependències: les fa explícites i evita que cada objecte hagi de construir-les.

3. Injecció de dependències: DI

La **injecció de dependències** és la tècnica que empram perquè un objecte rebi els col·laboradors que necessita. En els exemples de Spring, el contenidor els proporciona quan crea i configura el bean.

IoC és el principi general; DI és la manera concreta de connectar els objectes que treballarem. Podem aplicar DI també amb Java sense Spring: una classe pot rebre una dependència pel constructor encara que els objectes es construeixin manualment.

3.1. Programar contra una interfície

Definim el comportament que necessita el cotxe:

```
public interface Motor {
    String engega();
}
```

Les implementacions aporten aquest comportament de maneres diferents:

```
@Component
public class MotorBenzina implements Motor {
    @Override
    public String engega() {
        return "Motor benzina engega";
    }
}

@Component
@Primary
public class MotorElectric implements Motor {
    @Override
    public String engega() {
        return "Motor elèctric engega";
    }
}
```

Cotxe pot dependre de Motor, perquè només necessita el contracte `engega()`. Les anotacions declaren les implementacions com a components de Spring; més endavant precisarem què és un bean.

3.2. DI per constructor

La injecció per constructor proporciona les dependències en el moment de construir l'objecte. Primer podem veure una variant simplificada sense qualificador ni GPS:

```
@Component
public class Cotxe {
    private final Motor motor;

    public Cotxe(Motor motor) {
        this.motor = motor;
    }

    public String engega() {
        return "Engegant motor ->" + motor.engega();
    }
}
```

Spring resol un bean de tipus `Motor` i el passa al constructor. Com que `MotorElectric` està marcat amb `@Primary`, en aquest exemple és l'opció per defecte.

La injecció per constructor és la forma que prioritizam per a dependències obligatòries. La classe declara què necessita i conserva la referència en un camp final. Aquest camp no es pot reassignar després de construir el cotxe; això no implica que l'objecte `Motor` sigui immutable.

Si una classe té un únic constructor, Spring el pot utilitzar sense `@Autowired`. Al codi treballat a classe hi apareix l'anotació per fer visible la injecció:

```
@Autowired
public Cotxe(Motor motor) {
    this.motor = motor;
}
```

Amb un únic constructor, les dues formes tenen el mateix propòsit. Una dependència obligatòria que no es pot resoldre impedeix crear correctament el bean.

3.3. Seleccionar una implementació: `@Primary` i `@Qualifier`

Quan hi ha més d'un bean compatible amb el tipus que volem injectar, hem d'establir quin correspon. No podem confiar que Spring triï arbitràriament una implementació.

`@Primary` marca un candidat preferent per a una dependència individual sense una selecció més específica. No elimina els altres beans del contenidor.

`@Qualifier` concreta la selecció en un punt d'injecció. En l'exemple del cotxe:

```
@Autowired
public Cotxe(@Qualifier("motorBenzina") Motor motor) {
    this.motor = motor;
}
```

Aquest constructor demana el candidat identificat amb `motorBenzina`, encara que el motor elèctric sigui `@Primary`. En aquest exemple, Spring assigna per defecte el nom `motorBenzina` al component `MotorBenzina`.

Declaració	Dependència seleccionada en aquest exemple
<code>Cotxe(Motor motor)</code>	<code>MotorElectric</code> , perquè és <code>@Primary</code> .
<code>Cotxe(@Qualifier("motorBenzina") Motor motor)</code>	<code>MotorBenzina</code> .

Si hi ha diversos candidats sense una selecció que resolgui l'ambigüitat, la injecció falla. Les anotacions seleccionen beans compatibles; no creen implementacions que no s'hagin registrat.

3.4. DI per setter i dependències opcionals

La injecció per setter proporciona una dependència després de construir l'objecte. És útil quan la classe pot funcionar sense aquell col·laborador o amb un comportament per defecte.

El cotxe necessita un motor, però pot conduir sense GPS:

```
@Component
public class Gps {
    public String getPosition() {
        return "GPS 0.0N, 0.0E";
    }
}
```

```

@Component
public class Cotxe {
    private final Motor motor;
    private Gps gps;

    @Autowired
    public Cotxe(@Qualifier("motorBenzina") Motor motor) {
        this.motor = motor;
    }

    @Autowired(required = false)
    public void setGps(Gps gps) {
        this.gps = gps;
    }

    public String engega() {
        String resultat = "Engegant motor ->" + motor.engega();
        if (gps != null) {
            resultat += " | " + gps.getPosition();
        }
        return resultat;
    }
}

```

Si hi ha un bean Gps, Spring invoca `setGps(...)`. Si no hi és, `required = false` permet ometre aquesta injecció i el camp conserva el valor inicial null. `engega()` contempla tots dos casos. Aquesta és la configuració de Cotxe emprada a classe: component al paquet domain, motor de benzina seleccionat explícitament i GPS opcional.

En el projecte actual, Gps duu `@Component` i, per tant, s'injecta. Per observar l'alternativa sense GPS, podem retirar temporalment aquesta anotació i reiniciar; després la restauram. `required = false` permet l'absència, però no resol una ambigüitat entre diversos candidats.

El servei rep el cotxe i coordina l'operació de conduir:

```

@Service
public class CotxeService {
    private final Cotxe cotxe;

    @Autowired
    public CotxeService(Cotxe cotxe) {
        this.cotxe = cotxe;
    }

    public String conduir() {
        String resultat = "Conduint cotxe.\n" + cotxe.engega();
        return resultat;
    }
}

```

La ruta `/cotxe` d'`Unitat1Controller` crida `CotxeService.conduir()`, que delega en `Cotxe.engega()`. Així distingim l'objecte de domini del servei que l'utilitza.

Un setter no fa que una dependència sigui opcional per si mateix. Un setter amb `@Autowired` sense `required = false` continua requerint una dependència resoluble. En aquests apunts fem explícita l'opcionalitat al setter; el motor continua sent obligatori al constructor.

Un setter permet reassignar la dependència mitjançant una crida posterior, però això no significa que Spring la canviï automàticament mentre l'aplicació funciona.

Aspecte	Constructor	Setter
Moment de la injecció	Durant la construcció.	Després de la construcció.
Ús que prioritizam	Dependències obligatòries.	Dependències opcionals amb un comportament alternatiu definit.
Referència	Pot ser final.	Ha de permetre l'assignació posterior.
Precaució	Declarar les dependències que la classe necessita.	Configurar l'opcionalitat i contemplar l'absència.

3.5. La classe es pot provar sense arrencar Spring

Com que el cotxe rep una interfície, podem proporcionar-li un motor de prova amb Java:

```
Motor motorDeProva = () -> "Motor de prova engega.";
Cotxe cotxe = new Cotxe(motorDeProva);
String resultat = cotxe.engega();
```

La lambda implementa l'únic mètode de Motor. Aquí hem creat el cotxe manualment: no és un bean de Spring i Spring no hi injectarà automàticament el GPS. L'exemple mostra que la lògica del cotxe pot treballar amb una implementació alternativa sense canviar la classe.

4. Beans i contenidor de Spring

4.1. Què és un bean?

Un **bean** és un objecte gestionat pel contenidor IoC de Spring. La classe defineix el tipus i el comportament; el bean és la instància que Spring crea i configura segons una definició.

No tots els objectes Java de l'aplicació han de ser beans. Si una classe crea un objecte auxiliar amb `new`, aquest objecte no passa automàticament a estar gestionat per Spring. La presència d'una anotació a la seva classe tampoc no fa que una instància construïda manualment sigui gestionada pel contenidor.

En aquesta aplicació declaram beans de dues maneres: amb classes anotades que Spring detecta, o amb mètodes `@Bean` de configuració. Desenvoluparem aquesta segona forma al final dels apunts.

4.2. Components i estereotips

Anotació	Responsabilitat que expressa
<code>@Component</code>	Component genèric de l'aplicació.
<code>@Service</code>	Servei que executa operacions i lògica de negoci.
<code>@Repository</code>	Component d'accés a dades.
<code>@Controller</code>	Controlador web, habitualment per preparar vistes.
<code>@RestController</code>	Controlador que retorna el contingut de la resposta HTTP.

Els estereotips permeten detectar components i expressen la funció de les classes. Alguns tenen també comportaments específics: no són simplement etiquetes intercanviables.

L'anotació no programa la responsabilitat de la classe. Marcar un controlador no justifica posar-hi tota la lògica de negoci: el controlador rep la petició i delega al servei.

4.3. La connexió dels beans

Quan Spring ha de crear un bean, consulta les dependències declarades i obté els beans compatibles per proporcionar-les-hi. Aquesta resolució pot implicar crear altres objectes abans de completar el primer.

Per exemple, ja hem vist aquesta cadena de col·laboració:

```
Unitat1Controller -> CotxeService -> Cotxe -> MotorBenzina
                  -> Gps
```

Cada classe rep els col·laboradors que necessita. Les crides als mètodes continuen sent crides Java ordinàries; Spring ha preparat els objectes i les seves connexions.

Cal separar **la preparació dels objectes** de **l'atenció d'una petició**. Spring pot construir i connectar aquests beans a l'arrencada; rebre `/cotxe` fa executar els mètodes sobre els objectes preparats, sense tornar a injectar-los a cada crida.

5. Scopes: quantes instàncies hi ha i quan es reutilitzen

L'**scope**, o abast, defineix com es creen i comparteixen les instàncies d'un bean dins un context determinat. Per entendre'l hem de separar dues preguntes: quan demanam un objecte al contenidor i durant quant de temps conservam la referència que hem rebut.

Scope	Instàncies i abast	Finalització
singleton	Una instància per definició de bean i contenidor. És el valor per defecte.	Normalment, en tancar el context.
prototype	Una instància nova cada vegada que es demana el bean.	El codi que l'utilitza és responsable dels recursos que calgui alliberar.
request	Una instància per petició HTTP quan es necessita el bean.	En acabar la petició.
session	Una instància per sessió HTTP quan es necessita el bean.	En finalitzar la sessió.
application	Una instància associada al context de l'aplicació web, el ServletContext.	En finalitzar el context web.

Els tres darrers requereixen un context web. singleton es refereix al contenidor de Spring, mentre que application es refereix al ServletContext; aquesta distinció importa si hi ha diversos contextos de Spring dins una aplicació web.

5.1. Singleton: compartir la mateixa instància

En els exemples, els serveis són singleton per defecte. També ho és aquest comptador:

```
@Component
public class Contador {
    private AtomicInteger i = new AtomicInteger();

    public int next() {
        return i.incrementAndGet();
    }
}
```

Dos serveis reben el comptador per constructor:

```

@Service
public class ContadorServiceA {
    private final Contador counter;

    @Autowired
    public ContadorServiceA(Contador counter) {
        this.counter = counter;
    }

    public String print() {
        return "A " + counter.next();
    }
}

@Service
public class ContadorServiceB {
    private final Contador counter;

    @Autowired
    public ContadorServiceB(Contador counter) {
        this.counter = counter;
    }

    public String print() {
        return "B " + counter.next();
    }
}

```

Tots dos apunten a la mateixa instància de Contador. En una execució sense altres crides, invocar primer A.print() i després B.print() produeix A 1 i B 2. La ruta /singleton concatena aquests textos directament: la primera resposta és A 1B 2. La petició següent continua la seqüència i permet observar l'estat compartit.

Un singleton és adequat per a serveis compartits, repositoris i configuracions. Si hi manté estat mutable, aquest estat és compartit entre les peticions que hi accedeixen: Spring no el fa segur davant crides concurrents pel fet de declarar-lo singleton. El comptador utilitza AtomicInteger per fer atòmica l'operació d'increment.

5.2. Prototype: una instància per petició al contenidor

Un bean prototype es declara amb @Scope("prototype"). L'exemple dels tickets conserva una seqüència senzilla per distingir les instàncies:

```

@Component
@Scope("prototype")
public class Ticket {
    private static int seq = 0;
    private int id;

    public Ticket() {
        this.id = ++seq;
    }

    public int getId() {
        return id;
    }
}

```

Aquest comptador estàtic és una simplificació per a la demostració seqüencial. L'exemple dels carrets que veurem després utilitza UUIDs.

```

@Service
public class TicketService {
    private final Ticket ticket1;
    private final Ticket ticket2;

    @Autowired
    public TicketService(Ticket ticket1, Ticket ticket2) {
        this.ticket1 = ticket1;
        this.ticket2 = ticket2;
    }

    public String printTickets() {
        return "ticket1 " + ticket1.getId()
            + " ticket2 " + ticket2.getId();
    }
}

```

Spring resol dos tickets en construir TicketService: com que són prototype, tenen IDs diferents. Si Ticket fos singleton, tots dos paràmetres rebrien la mateixa instància.

Però TicketService és singleton i conserva els dos tickets en els seus camps. Repetir /prototype no executa de nou el constructor del servei: continuem veient els mateixos dos IDs.

Prototype no significa un objecte nou a cada crida d'un mètode ni a cada petició HTTP.

Significa un objecte nou cada vegada que el contenidor rep una petició d'aquell bean.

5.3. Demanar el prototype quan cal: ObjectProvider i una factory

L'exemple del carret fa visible aquest problema:

```

@Component
@Scope("prototype")
public class ShoppingCart {
    private final UUID id = UUID.randomUUID();

    public UUID getId() {
        return id;
    }
}

```

Volem obtenir un carret nou cada vegada que una operació el necessita. Per fer la petició al contenidor en aquell moment, injectam un ObjectProvider<ShoppingCart> i invocam getObject().

Encapsulam aquest mecanisme en una **factory**, una classe que proporciona els objectes que necessitam:

```

@Component
public class ShoppingCartFactory {
    public final ObjectProvider<ShoppingCart> provider;

    @Autowired
    public ShoppingCartFactory(ObjectProvider<ShoppingCart> provider) {
        this.provider = provider;
    }

    public ShoppingCart createCart() {
        return provider.getObject();
    }
}

```

El servei rep la factory i obté un carret nou quan comença l'operació:

```

@Service
public class ShoppingCartService {
    private final ShoppingCartFactory cartFactory;

    @Autowired
    public ShoppingCartService(ShoppingCartFactory cartFactory) {
        this.cartFactory = cartFactory;
    }

    public String startCheckout() {
        ShoppingCart shoppingCart = cartFactory.createCart();
        return shoppingCart.getId().toString();
    }
}

```

El controlador permet comparar la injecció directa amb el recorregut pel servei i la factory:

```

@RestController
@RequestMapping("/cart")
public class CartController {
    private final ShoppingCart directCart;
    private final ShoppingCartService shoppingCartService;

    @Autowired
    public CartController(ShoppingCart directCart,
        ShoppingCartService shoppingCartService) {
        this.directCart = directCart;
        this.shoppingCartService = shoppingCartService;
    }

    @GetMapping("/direct")
    public String direct() {
        return "Cart injectat directament: " + directCart.getId();
    }

    @GetMapping("/factory")
    public String factory() {
        return "Cart creat amb factory: " + shoppingCartService.startCheckout();
    }
}

```

Ruta	Què esperam en repetir-la sense reiniciar l'aplicació?	Per què?
/cart/direct	El mateix UUID.	El controlador singleton conserva el carret rebut al constructor.
/cart/factory	UUIDs diferents.	Cada crida a createCart() demana un altre bean prototype.

La ruta /cart/factory segueix CartController → ShoppingCartService → ShoppingCartFactory. A /cart/direct, el controlador rep el carret directament per simplificar la comparació; en aquesta demostració no hi afegim un servei intermedi.

prototype significa que Spring crea una nova instància cada vegada que se li demana el bean. ObjectProvider permet fer aquesta petició en el moment que la necessitam.

La factory pot ser singleton: conserva el proveïdor, no un carret per reutilitzar. Cada invocació de createCart() fa una petició nova. El proveïdor respecta l'scope del bean; la creació d'instàncies diferents correspon a @Scope("prototype").

El camp provider és public final al projecte. La resta del codi utilitza createCart(); declarar el camp private final seria una millora d'encapsulació que no canviaria el comportament de l'exemple.

Aquest ShoppingCart només conté un UUID per distingir instàncies. No conserva productes ni representa un carret d'usuari entre peticions: cada crida a la factory en crea un de nou. Per conservar una compra caldria dissenyar on guardar-ne l'estat, per exemple en una sessió o amb persistència. A més, ObjectProvider no força sempre una instància nova: si canviàssim el bean a singleton, getObject() retornaria la instància compartida.

Aplicació al cas dels informes

Com a trasllat a la pràctica d'informes, la mateixa idea serveix per a ReportJob, un treball prototype que ha de ser nou a cada generació. ReportJob i ReportJobFactory no formen part de unitat1_2627: són el disseny que aplicam a aquella pràctica. Els serveis de generació són singleton; injectar-hi un treball directament faria que el conservassin.

La factory equivalent rep ObjectProvider<ReportJob> i el seu mètode createJob() retorna provider.getObject(). El servei rep la factory per constructor i, dins cada generació, obté el treball amb:

```
ReportJob job = reportJobFactory.createJob();
```

La declaració de ReportJob com a prototype és la que garanteix un treball nou. La factory concentra la petició a Spring i permet que els serveis no utilitzin directament ObjectProvider.

5.4. Request: una instància per petició HTTP

Un bean amb scope request manté la seva instància durant una petició HTTP. Si diferents components demanen aquell bean dins la mateixa petició, comparteixen la instància. Una altra petició disposa d'una altra instància quan la necessita.

```
@Component
@Scope(value = "request", proxyMode = ScopedProxyMode.TARGET_CLASS)
public class RequestBean {
    private final String id;

    public RequestBean() {
        this.id = UUID.randomUUID().toString();
    }

    public String getId() {
        return id;
    }
}

@Service
@Scope("singleton")
public class RequestService {
    private final RequestBean requestBean;

    @Autowired
    public RequestService(RequestBean requestBean) {
        this.requestBean = requestBean;
    }

    public String getRequestBeanId() {
        return requestBean.getId();
    }
}
```

El servei declara explícitament @Scope("singleton"), tot i que ja és el valor per defecte.

L'scope de la dependència no canvia el del servei, que es pot crear quan encara no hi ha cap petició HTTP. Per poder injectar-hi una dependència de petició, utilitzam un **proxy d'scope**: un objecte intermedi que delega les crides en el bean corresponent a la petició activa.

El servei conserva el proxy. El proxy no es reemplaça en el camp del servei a cada petició; quan invocam `getId()`, localitza la instància corresponent a la petició actual. A `/request`, peticions successives mostren UUIDs diferents. Accedir al bean fora d'una petició activa no disposa d'aquest scope.

`ScopedProxyMode.TARGET_CLASS` indica que el proxy es basa en la classe del bean. Per permetre aquesta intermediació, en aquests exemples la classe i el mètode que cridam no han de ser final. El camp `private final RequestBean requestBean` sí que pot ser-ho: conserva la referència al proxy.

5.5. Session: una instància per sessió HTTP

Una sessió HTTP permet conservar estat entre diverses peticions d'un mateix client. En el cas habitual, el navegador envia una galeta amb l'identificador de sessió i el servidor reconeix la sessió corresponent.

```
@Component
@Scope(value = "session", proxyMode = ScopedProxyMode.TARGET_CLASS)
public class SessionBean {
    private final String id;

    public SessionBean() {
        this.id = UUID.randomUUID().toString();
    }

    public String getId() {
        return id;
    }
}

@Service
public class SessionService {
    private final SessionBean sessionBean;

    @Autowired
    public SessionService(SessionBean sessionBean) {
        this.sessionBean = sessionBean;
    }

    public String getSessionBeanId() {
        return sessionBean.getId();
    }
}
```

També necessitam un proxy perquè el servei singleton pugui delegar en la instància de la sessió activa. A `/session`, les peticions de la mateixa sessió mostren el mateix UUID; una sessió diferent en mostra un altre.

L'UUID mostrat és un valor creat pel nostre `SessionBean`, no l'identificador de la galeta de sessió. Ens permet reconèixer la instància del bean; el servidor gestiona separatament la identificació de la sessió HTTP.

Per comparar sessions, podem emprar navegadors diferents o perfils que no comparteixin galetes. Obrir una altra pestanya del mateix navegador normalment conserva la sessió. Una sessió no equival necessàriament a un usuari autenticat i pot acabar per caducitat o invalidació.

5.6. Application i criteri de tria

L'scope `application` associa el bean al `ServletContext` i el comparteix dins aquella aplicació web. El conservam com a part del repertori d'scopes, encara que els exemples de la unitat se centren en `singleton`, `prototype`, `request` i `session`.

Per triar l'scope, pensam en qui ha de compartir l'objecte: tota l'aplicació, una operació independent, una petició o una sessió. Els serveis solen ser `singleton` i han d'evitar conservar dades temporals d'un usuari en camps compartits. Un treball independent pot ser `prototype`, i les dades específiques d'una sessió poden requerir un bean de sessió.

L'scope determina com es comparteix una instància; no substitueix el disseny de les responsabilitats ni resol automàticament els problemes de concurrència.

6. Inicialització diferida: @Lazy

6.1. Inicialització immediata i diferida

Per defecte, Spring crea els beans singleton no diferits quan inicialitza el context. Aquest comportament s'anomena **eager initialization**: la creació i la preparació dels objectes formen part de l'arrencada.

Si un bean necessita temps per carregar dades, configurar un client extern o preparar recursos, aquest cost afecta l'arrencada encara que no l'utilitzem immediatament. A canvi, molts errors de creació i de connexió de dependències es detecten abans d'atendre peticions.

La **lazy initialization**, o inicialització diferida, permet ajornar la creació d'un bean fins que sigui necessari. Per entendre el comportament hem de distingir la configuració del bean de la configuració d'un punt d'injecció.

6.2. Exemple sense inicialització diferida

Simulam un component que necessita cinc segons per construir-se:

```
@Component
public class HeavyBean {
    public HeavyBean() {
        System.out.println(">> Constructor de HeavyBean: Inicialitzant...");
        try {
            Thread.sleep(5000);
        } catch (InterruptedException ignored) {
            ignored.printStackTrace();
        }
        System.out.println(">> HeavyBean inicialitzat");
    }

    public String process() {
        return "HeavyBean process completat!";
    }
}

@Service
public class HeavyService {
    private final HeavyBean heavyBean;

    @Autowired
    public HeavyService(HeavyBean heavyBean) {
        this.heavyBean = heavyBean;
    }

    public String doWork() {
        return heavyBean.process();
    }
}
```

En aquesta variant sense @Lazy, Spring crea HeavyBean durant l'arrencada. El retard és al constructor de **HeavyBean**, i HeavyService necessita aquest objecte per completar la seva construcció. La crida posterior a /lazy, que delega en doWork(), ja troba el component preparat: el nom de la ruta, per si sol, no activa la inicialització diferida.

L'espera fa visible el temps de creació. El catch reproduïx el projecte, però només imprimeix l'error i continua com si la preparació s'hagués completat. Com a millora per a codi real, caldria

restaurar la interrupció amb `Thread.currentThread().interrupt()` i interrompre la inicialització, per exemple llançant una `IllegalStateException` amb la causa.

6.3. @Lazy al bean

Podem marcar la classe perquè el contenidor no la preinstancii pel simple fet de ser singleton:

```
@Component
@Lazy
public class HeavyBean {
    // Mateix constructor i mateix mètode process() de l'exemple anterior.
}
```

També es pot posar `@Lazy` en un mètode `@Bean`.

Ara bé, si un singleton no diferit demana `HeavyBean` directament al constructor, Spring ha de crear-lo per satisfer aquella dependència durant l'arrencada. **Marcar el bean amb `@Lazy` no impedeix que una dependència directa el faci necessari abans del que esperàvem.**

6.4. @Lazy al punt d'injecció

En aquest cas, `@Lazy` demana un proxy que retarda la resolució de la dependència:

```
@Service
public class HeavyService {
    private final HeavyBean heavyBean;

    @Autowired
    public HeavyService(@Lazy HeavyBean heavyBean) {
        this.heavyBean = heavyBean;
    }

    public String doWork() {
        return heavyBean.process();
    }
}
```

El servei pot construir-se amb el proxy sense necessitar encara la instància real. Quan es crida `process()`, el proxy resol el bean real i hi delega.

Si `HeavyBean` continua declarat com a singleton no diferit, el contenidor encara el pot crear a l'arrencada pel seu propi procés de preinstanciació. Per això l'exemple del projecte combina **`@Lazy` a `HeavyBean` i al paràmetre del constructor de `HeavyService`.**

Lloc on es posa <code>@Lazy</code>	Què configura?
Classe del bean o mètode <code>@Bean</code>	Ajornar la preinstanciació d'aquell bean.
Punt d'injecció	Injectar un proxy que resol aquella dependència quan s'utilitza.

La primera anotació evita crear el component per endavant; la segona evita que la construcció del servei el demani immediatament. La combinació manté `HeavyService` disponible a l'arrencada i ajorna **`HeavyBean`** fins a l'ús real.

En la demostració, sempre que cap altre component en forci la creació:

1. A l'arrencada no apareix el missatge del constructor de `HeavyBean`.
2. La primera petició a `/lazy` en provoca la creació i mostra el retard.
3. Les peticions següents reutilitzen el mateix singleton i no repeteixen la inicialització.

El retard ha d'estar a la inicialització. Si posàssim `Thread.sleep(...)` dins `process()`, cada invocació repetiria l'espera i estaríem mostrant el cost d'una operació, no el cost de crear el bean.

6.5. Inicialització diferida global amb Spring Boot

El code along utilitza `@Lazy` al bean i al punt d'injecció. Com a variant de configuració, podem aplicar el mateix criteri de manera global; el component Warmup següent és un exemple per explicar-ne una excepció.

Ampliació: ni la propietat següent ni Warmup són presents al projecte actual. No els necessitam per reproduir /lazy.

Podem habilitar la inicialització diferida de manera global a `src/main/resources/application.properties`:

```
spring.main.lazy-initialization=true
```

Spring Boot habilita la inicialització diferida de manera general. No hem d'interpretar-ho com que qualsevol component de l'aplicació esperarà necessàriament una petició: alguns components són necessaris per arrencar i altres poden ser demanats durant aquest procés.

Podem excloure'n un component que volem inicialitzar per endavant:

```
@Component
@Lazy(false)
public class Warmup {
    public Warmup() {
        System.out.println(">> Warmup preparat a l'arrencada");
    }
}
```

6.6. Quan és útil i què canvia?

La inicialització diferida és útil per a components costosos que només s'utilitzen ocasionalment. No elimina el cost de preparar-los: el desplaça al moment en què es necessiten.

Aquest canvi té conseqüències: una primera petició pot ser més lenta i alguns errors de configuració poden aparèixer al primer ús. Si diverses peticions arriben mentre es crea un singleton, poden haver d'esperar que se'n completi la inicialització.

Els prototypes ja es creen quan es demanen. No necessiten `@Lazy` al bean per evitar la preinstanciació habitual dels singletons. En el cas dels informes, el que volem és demanar un treball nou a cada generació amb la factory; en el cas de HeavyBean, volem ajornar la creació d'una instància compartida que després reutilitzarem.

7. Cicle de vida: @PostConstruct i @PreDestroy

7.1. De la construcció a la finalització

Spring no es limita a invocar un constructor. També prepara les dependències i pot executar mètodes d'inicialització i de finalització.

Per als exemples d'aquesta unitat, podem descriure'n el cicle així:

1. **Construcció:** Spring resol els arguments del constructor i crea l'objecte.
2. **Injecció posterior:** configura les dependències que s'injecten mitjançant setters o altres punts d'injecció.
3. **Inicialització:** executa els callbacks d'inicialització, com @PostConstruct, amb les dependències ja proporcionades.
4. **Ús:** l'aplicació invoca els mètodes del bean.
5. **Finalització:** quan acaba l'abast gestionat, Spring pot executar els callbacks de destrucció, com @PreDestroy.

La injecció per constructor forma part de la construcció: les seves dependències no esperen una fase posterior. El contenidor també aplica components interns anomenats BeanPostProcessor, que poden intervenir en la preparació del bean; no els implementarem en aquesta unitat.

7.2. Exemple amb un generador d'informes

Utilitzam aquesta classe per observar el cicle de vida:

```
import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.springframework.stereotype.Component;

@Component
public class ReportGenerator {
    public ReportGenerator() {
        System.out.println(">> Constructor ReportGenerator");
    }

    @PostConstruct
    public void init() {
        System.out.println(">> Inicialitzant recursos de ReportGenerator");
    }

    @PreDestroy
    public void cleanup() {
        System.out.println(">> Alliberant recursos de ReportGenerator");
    }

    public String generate() {
        return "Informe generat!";
    }
}
```

Les anotacions provenen de jakarta.annotation. Els missatges fan visible quan s'executa cada mètode; en aquest exemple no hi ha una connexió o un fitxer real per obrir i tancar.

El servei rep el generador per constructor:

```

@Service
public class ReportService {
    private final ReportGenerator reportGenerator;

    @Autowired
    public ReportService(ReportGenerator reportGenerator) {
        this.reportGenerator = reportGenerator;
    }

    public String generateReport() {
        return reportGenerator.generate();
    }
}

```

La ruta /report d'Unitat1Controller delega en ReportService.generateReport().

Moment	Comportament de l'exemple
Arrencada	Es construeix ReportGenerator i després s'executa init().
Petició a /report	Es crida generate() i es retorna Informe generat!.
Peticions següents	Es reutilitza el bean; init() no es repeteix per informe.
Tancament ordenat del context	S'executa cleanup().

@PostConstruct s'executa per instància inicialitzada; no és una anotació per executar codi abans de cada operació. @PreDestroy s'associa a la finalització gestionada del bean, no al final de cada crida.

En aquest exemple, ReportGenerator és una classe concreta per mostrar aquests callbacks. A la pràctica d'informes, el mateix nom identifica una interfície amb diverses implementacions. Cal distingir la funció que té en cada cas.

7.3. Què podem fer als mètodes de cicle de vida?

Els casos següents donen context a les dues anotacions. No cal instal·lar aquestes tecnologies per seguir el code along: ReportGenerator només escriu missatges. La idea que cal retenir és preparar un recurs quan el component s'inicialitza i alliberar-lo quan acaba la seva vida, si aquest component n'és responsable. Un @PostConstruct s'executa durant la inicialització del bean; no garanteix que tota l'aplicació ja estigui preparada per atendre peticions.

Casos d'ús de @PostConstruct

@PostConstruct permet preparar un component amb les dependències ja injectades. Alguns casos d'ús reals són:

- **Preparar una memòria cau amb Redis.** Redis és un magatzem de dades que treballa principalment en memòria i s'utilitza sovint com a memòria cau per consultar informació ràpidament. Lettuce és una biblioteca Java per comunicar-se amb Redis. Un servei pot carregar un catàleg o unes dades de consulta freqüent a Redis abans d'atendre operacions. Si el component gestiona directament el client Lettuce, pot preparar la connexió amb RedisClient.connect() i utilitzar-la per executar les operacions de càrrega.
- **Preparar un productor de missatges amb Kafka.** Kafka és una plataforma que permet publicar, conservar i consumir fluxos d'esdeveniments entre aplicacions; per exemple, comunicar que s'ha creat una comanda perquè altres serveis hi reaccionin. Un component que publica esdeveniments pot construir un KafkaProducer amb els servidors, els

serialitzadors i la configuració que ha rebut. El productor es conserva per reutilitzar-lo quan s'hagin d'enviar missatges, en lloc de crear-lo per a cada enviament.

- **Preparar connexions i cues amb RabbitMQ.** RabbitMQ és un servidor de missatgeria que rep missatges i els distribueix a cues perquè altres components els processin; per exemple, una cua de correus pendents d'enviar. Un component pot obrir una connexió amb `ConnectionFactory.newConnection()`, obtenir un canal amb `createChannel()` i declarar la cua que necessita amb `queueDeclare(...)`. Aquesta preparació permet que les operacions posteriors utilitzin la infraestructura de missatgeria.
- **Preparar tasques en segon pla.** Són feines que s'executen fora del fil que atén la petició, com enviar correus, generar informes o processar fitxers, i també feines periòdiques com netejar dades temporals. Un pool de fils permet repartir aquesta feina entre un conjunt de fils reutilitzables. Un servei pot crear un pool amb `Executors.newFixedThreadPool(...)` per executar-la, o un `ScheduledExecutorService` amb `Executors.newScheduledThreadPool(...)` per programar tasques periòdiques, com revisar treballs pendents o renovar una informació local.
- **Carregar credencials, claus o certificats.** Les credencials permeten autenticar-se davant un servei; les claus criptogràfiques s'utilitzen per xifrar o signar, i els certificats vinculen una identitat amb una clau pública. Un component de seguretat pot llegir un fitxer de credencials o carregar un magatzem de claus amb `KeyStore.load(...)` per preparar la signatura de documents o la verificació de certificats. El flux de lectura es tanca després de carregar les dades, habitualment amb `try-with-resources`; no cal deixar el fitxer obert durant tota la vida del bean.
- **Obtenir secrets d'un Vault.** HashiCorp Vault és un servei per guardar i gestionar secrets, com contrasenyes, tokens o claus, i controlar quines aplicacions hi poden accedir. Si les credencials es guarden a Vault, un servei pot emprar el `VaultTemplate` injectat i `read(...)` per recuperar un secret necessari per preparar un client extern. És una lectura inicial; renovar un secret amb caducitat requereix també un mecanisme de renovació.
- **Preparar recursos i validar la configuració.** Un client HTTP permet fer peticions a altres serveis, i un SDK és un conjunt d'eines i biblioteques que facilita utilitzar una plataforma externa. Un generador pot carregar plantilles, un servei pot preparar un client HTTP o un SDK, i un component pot comprovar que els valors injectats permeten funcionar correctament abans d'acceptar operacions.

Casos d'ús de @PreDestroy

@PreDestroy permet finalitzar la feina i alliberar els recursos que el component ha gestionat. Alguns casos d'ús reals són:

- **Tancar la connexió amb Redis.** Amb Lettuce, un component que és propietari de la connexió i del client pot executar `connection.close()` i `redisClient.shutdown()` per alliberar les connexions i els recursos del client.
- **Finalitzar un productor de Kafka.** `KafkaProducer.close()` permet tancar el productor i els seus recursos. Quan cal esperar explícitament els enviaments pendents abans de continuar, `flush()` permet esperar la finalització dels missatges ja enviats al productor.
- **Tancar canals i connexions amb RabbitMQ.** Si el component els ha oberts i els gestiona, pot executar `channel.close()` i `connection.close()` en finalitzar. Les connexions es reutilitzen durant el funcionament; no s'obren i tanquen per cada missatge.

- **Aturar els pools de fils i les tasques periòdiques.** Un `ExecutorService` es pot aturar amb `shutdown()`, que deixa d'acceptar tasques noves i permet acabar les ja presentades. `awaitTermination(...)` permet esperar-ne la finalització amb un límit de temps. També es poden cancel·lar tasques periòdiques conservant-ne el `ScheduledFuture` i invocant `cancel(...)`.
- **Retirar una instància del registre de serveis d'Eureka.** Eureka és un registre on les instàncies dels serveis anuncien la seva adreça perquè altres serveis les puguin localitzar, sense haver de conèixer per endavant on s'executen. El client de Netflix ofereix un exemple real: `DiscoveryClient.shutdown()` està anotat amb `@PreDestroy`, cancel·la tasques de manteniment i, segons la configuració, envia la petició de baixa al servidor Eureka.
- **Finalitzar recursos propis.** Un component pot tancar un client HTTP, retirar una subscripció, eliminar fitxers temporals o desar un estat temporal que s'ha de recuperar a la següent arrencada.

Aquests casos mostren feines que es poden vincular al cicle de vida; no impliquen que totes aquestes biblioteques utilitzin internament les dues anotacions. Quan Spring o una integració ja gestiona un client, una connexió o un executor, aprofitam aquest mecanisme i evitam inicialitzar-lo o tancar-lo una segona vegada des d'un altre bean. Algunes integracions coordinen l'arrencada i l'aturada amb altres mecanismes de cicle de vida, com passa amb la gestió completa del client d'Eureka.

Cada component ha d'alliberar els recursos dels quals és responsable. No ha de tancar arbitràriament una connexió o un recurs compartit que gestiona un altre component.

Els callbacks de finalització requereixen que el context es tanqui correctament. No s'ha de donar per garantit `@PreDestroy` davant una terminació forçada del procés, i no s'ha de confiar només en aquest mètode per conservar informació que no es pot perdre.

7.4. El cas dels prototypes

Spring construeix el prototype, hi proporciona les dependències i executa els callbacks d'inicialització aplicables. Després el lliura al codi que l'ha demanat.

Spring no gestiona automàticament la destrucció dels prototypes que ha lliurat. Si un prototype manté recursos que s'han de tancar, el codi que l'utilitza n'ha de preveure la neteja. No podem esperar que tots els seus `@PreDestroy` s'executin quan tanquem l'aplicació.

Aquest límit també s'aplica als treballs obtinguts amb `ReportJobFactory`: la factory permet demanar instàncies a Spring, però no n'amplia la gestió del cicle de vida.

8. Configuració explícita: @Configuration i @Bean

8.1. Quan no podem anotar la classe

Fins ara hem declarat components anotant classes pròpies. Però sovint volem injectar un objecte d'una biblioteca externa o de la biblioteca estàndard de Java. No podem modificar aquestes classes per posar-hi @Component.

També pot interessar-nos separar la configuració d'un objecte de les classes que l'utilitzen. En lloc de construir-lo dins cada servei, el declaram una vegada i el proporcionam per DI.

@Configuration identifica una classe de configuració. Dins aquesta classe, @Bean declara que el resultat d'un mètode s'ha de registrar com a bean.

8.2. Exemple: formatar l'hora del servidor

Volem que una petició a /hora retorni la data i l'hora locals del servidor amb el patró dd/MM/yyyy HH:mm:ss.

L'objecte que proporciona el format és java.time.format.DateTimeFormatter. El declaram a HoraConfiguration:

```
import java.time.format.DateTimeFormatter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class HoraConfiguration {
    @Bean
    public DateTimeFormatter formatDataHora() {
        return DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss");
    }
}
```

La classe va al paquet config, dins l'àmbit de cerca de components. Spring incorpora aquesta configuració i registra el formatador. Per defecte, el bean es diu formatDataHora, com el mètode, i té scope singleton.

El resultat del mètode @Bean és una instància de DateTimeFormatter. HoraConfiguration declara com obtenir l'objecte que volem injectar.

El patró utilitza dd per al dia del mes, MM per al mes, yyyy per a l'any, HH per a l'hora en format de 24 hores, mm per als minuts i ss per als segons. Les majúscules i les minúscules importen: MM i mm tenen significats diferents.

DateTimeFormatter és immutable i segur per compartir entre fils. En aquest cas és adequat reutilitzar-lo com a singleton.

8.3. Injectar el formatador en el rellotge

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class RellotgeServidor {
    private final DateTimeFormatter formatDataHora;

    @Autowired
    public RellotgeServidor(DateTimeFormatter formatDataHora) {
        this.formatDataHora = formatDataHora;
    }

    public String dataHoraActual() {
        return LocalDateTime.now().format(formatDataHora);
    }
}
```

RellotgeServidor és una classe pròpia i la podem anotar amb `@Component`. Rep el formatador per constructor, igual que el cotxe rebia el motor. No necessita saber com s'ha construït ni quin mètode de configuració l'ha declarat.

`LocalDateTime.now()` consulta la data i l'hora locals amb la zona per defecte del servidor. L'exemple no converteix l'hora a la zona del navegador ni envia informació de zona o desplaçament horari.

8.4. Servei i controlador

```
@Service
public class HoraService {
    private final RellotgeServidor rellotgeServidor;

    @Autowired
    public HoraService(RellotgeServidor rellotgeServidor) {
        this.rellotgeServidor = rellotgeServidor;
    }

    public String consultarHora() {
        return "Data i hora del servidor: "
            + rellotgeServidor.dataHoraActual();
    }
}

@RestController
public class Unitat1Contoller {
    private final HoraService horaService;

    // S'ometen les altres dependències del controlador.

    @Autowired
    public Unitat1Contoller(HoraService horaService) {
        this.horaService = horaService;
    }

    @GetMapping("/hora")
    public String hora() {
        return horaService.consultarHora();
    }

    // S'ometen les altres rutes.
}
```

Aquest fragment mostra només la dependència i la ruta de l'hora. A classe, `Unitat1Contoller` reuneix també les rutes del cotxe, els scopes, la inicialització diferida i els informes, amb totes les dependències al mateix constructor.

El recorregut d'una petició és:

```
GET /hora
-> Unitat1Contoller.hora()
-> HoraService.consultarHora()
-> RelotgeServidor.dataHoraActual()
-> LocalDateTime.now().format(formatDataHora)
-> text de resposta al navegador
```

Una resposta possible és Data i hora del servidor: 08/10/2026 15:30:00. El valor concret depèn del moment de la petició i de l'hora del servidor.

El controlador rep la petició; el servei coordina la consulta; el rellotge obté i formata la data i l'hora. La configuració proporciona el formatador que el rellotge necessita.

8.5. Què canvia respecte de `@Component`?

Aspecte	<code>@Component</code>	<code>@Bean</code> dins <code>@Configuration</code>
On es declara?	A la classe del component.	En un mètode de configuració.
Què registra?	Una instància de la classe detectada.	L'objecte que retorna el mètode.
Quan és útil?	Quan controlam la classe i la podem anotar.	Quan volem configurar-ne la creació o utilitzar una classe externa.
Exemple	<code>RelotgeServidor</code> .	<code>DateTimeFormatter</code> .

Totes dues formes produeixen beans que es poden injectar. El scope per defecte continua sent singleton, i podem aplicar-hi els conceptes d'inicialització i cicle de vida que hem estudiat.

Si canviem el patró a `HoraConfiguration` i reiniciam l'aplicació, canvia la representació de la data i l'hora sense haver de modificar `RelotgeServidor`, `HoraService` ni `Unitat1Contoller`. La configuració queda concentrada en un punt.

El bean compartit és el **formatador**, no el text d'una hora calculada durant l'arrencada.

`LocalDateTime.now()` s'executa a cada consulta; si calculàssim una data una sola vegada i la conservàssim, retornaria aquell instant en lloc de l'hora actual.

9. Relació entre els conceptes i els exemples

Els conceptes responen a preguntes diferents sobre els objectes de l'aplicació:

Pregunta	Concepte
Qui prepara els objectes i les seves connexions?	Contenedor IoC.
Com rep una classe els seus col·laboradors?	DI per constructor o setter.
Quina implementació d'una interfície rep?	Resolució per tipus, @Primary i @Qualifier.
Quina instància es comparteix i durant quant de temps?	Scope.
Com demanam un prototype en cada operació?	ObjectProvider encapsulat en una factory.
Quan assumim el cost de crear un bean?	Inicialització immediata o diferida.
Què s'executa després de la injecció i en finalitzar?	Callbacks de cicle de vida.
Com registram un objecte que no podem anotar?	@Configuration i @Bean.

9.1. Què podem observar al code along?

Ruta	Classes principals	Comportament que cal poder explicar
/cotxe	CotxeService, Cotxe, MotorBenzina, MotorElectric, Gps	El servei utilitza el cotxe; selecció de motor i comportament amb GPS o sense.
/singleton	Contador i els dos serveis	El recompte és compartit.
/prototype	Ticket, TicketService	Dos tickets diferents, conservats pel servei entre peticions.
/cart/direct	ShoppingCart, CartController	Es conserva el carret injectat al controlador.
/cart/factory	CartController, ShopingCartService, ShopingCartFactory	El controlador delega al servei, que demana un carret nou a la factory.
/request	RequestBean, RequestService	L'ID correspon a la petició activa.
/session	SessionBean, SessionService	L'ID es conserva dins la mateixa sessió.

Ruta	Classes principals	Comportament que cal poder explicar
/lazy	HeavyBean, HeavyService	El cost d'inicialització apareix al primer ús amb @Lazy al bean i al punt d'injecció.
/report	ReportGenerator, ReportService	Construcció, inicialització, ús i finalització són moments diferents.
/hora	HoraConfiguration, RellotgeServidor, HoraService, Unitat1Contoller	Un bean de configuració s'injecta i determina el format de la resposta.

Els exemples d'IDs i de temps permeten observar el resultat, però l'explicació ha de relacionar-lo amb el codi. Veure un UUID diferent, per si sol, no ens diu quin scope s'ha utilitzat: hem de saber on es crea l'objecte i quan es demana.

9.2. Distincions que hem de tenir clares

- Una classe anotada defineix un component; cada instància creada manualment no es converteix automàticament en un bean.
- Un singleton comparteix una instància per definició i contenidor, no necessàriament una instància universal per a qualsevol aplicació Java.
- Un prototype es crea quan es demana al contenidor; una referència ja injectada no es renova tota sola.
- Un setter pot ser obligatori. L'opcionalitat del GPS es configura i el codi contempla l'absència.
- @Lazy a la definició del bean i al punt d'injecció resolen dues parts diferents de l'ajornament.
- @PostConstruct prepara una instància després de la injecció, no abans de cada petició.
- @PreDestroy depèn de la finalització gestionada i no cobreix automàticament la destrucció dels prototypes.
- Un mètode @Bean registra l'objecte que retorna i en concentra la configuració.

9.3. Com comprovar els exemples

Aquesta és una pauta per revisar el projecte, amb la configuració original i sense altres peticions simultànies. Abans de cada comprovació, prediu el resultat i identifica la línia de codi que el justifica.

Comprovació	Resultat esperat i explicació que has de poder donar
Arrenca i visita / i /cotxe.	ho la mon; després, el text de conducció amb motor de benzina i GPS. El qualificador selecciona el motor i el setter rep el GPS disponible.
Visita /singleton dues vegades després d'arrencar.	A 1B 2 i A 3B 4: tots dos serveis comparteixen el comptador.
Visita /prototype dues vegades.	Dos IDs diferents entre si, però la mateixa parella a les dues respostes: el servei conserva els tickets.

Comprovació	Resultat esperat i explicació que has de poder donar
Repeteix <code>/cart/direct</code> i <code>/cart/factory</code> .	UUID estable a la primera ruta i variable a la segona. Localitza en quin moment es demana el carret a Spring.
Repeteix <code>/request</code> i <code>/session</code> ; després obre <code>/session</code> en un altre perfil de navegador.	L'UUID de petició canvia; el de sessió es conserva fins que uses una altra sessió.
Reinicia i visita <code>/lazy</code> dues vegades.	La primera crida espera aproximadament cinc segons i mostra els missatges de construcció; la segona reutilitza el bean.
Observa la consola des de l'arrencada, visita <code>/report</code> dues vegades i atura l'aplicació ordenadament.	Constructor i <code>init()</code> a l'arrencada, Informe generat! a cada resposta i <code>cleanup()</code> en tancar el context.
Visita <code>/hora</code> en dos moments separats per uns segons.	La data i l'hora s'actualitzen amb el mateix patró; el formatador es reutilitza.

Les rutes actuals permeten comparar peticions diferents. Per observar explícitament que un bean request es reutilitza **dins una mateixa petició**, una variant de prova és cridar `requestService.getRequestBeanId()` dues vegades dins el mètode `request()` i retornar tots dos valors: han de coincidir. En una petició posterior, canvia la parella. Aquesta variant no està implementada al projecte de referència.

9.4. Si el resultat no és l'esperat

Síntoma	Què convé revisar?
No es pot crear un bean perquè falta una dependència.	Que la implementació estigui registrada, dins la cerca de components i amb un tipus compatible.
Hi ha diversos candidats per a una dependència.	Que <code>@Primary</code> o el <code>@Qualifier</code> del punt d'injecció resolguin la selecció. Fer-la opcional no resol l'ambigüitat.
<code>/lazy</code> ja no mostra el retard.	Si ja s'ha utilitzat el bean des de l'última arrencada; després, les dues ubicacions de <code>@Lazy</code> i les dependències que en puguin forçar la creació.
<code>/session</code> canvia d'UUID a cada petició.	Si el client conserva i reenvia la galeta de sessió, si la sessió ha caducat o si s'ha reiniciat el servidor.

Una resposta diferent de la prevista s'ha d'explicar relacionant configuració, moment de creació i referències conservades. Canviar anotacions sense aquesta explicació no permet distingir un error d'una variació esperable de l'exemple.